COMPLETE NOTES ON GIT

---

1. Introduction to Git

What is Git?

Git is a **distributed version control system (DVCS)** used to track changes in source code during development.

📌 **Why Git?**

- Tracks changes in files
- Allows multiple people to collaborate
- Maintains history of project
- Easy rollback to previous versions

---

2. Git vs GitHub

Feature	Git	GitHub
Type	Tool	Platform
Usage	Version control	Code hosting
Works Offline	Yes	No
Example	Local repo	Remote repo

Git = software
GitHub (GitHub) = cloud platform

◈ 3. Git Architecture

Git has 3 main areas:

1. **Working Directory**
   - Where files are edited
2. **Staging Area (Index)**
   - Prepares files for commit
3. **Repository**
   - Stores committed files permanently

Installation

Linux (Kali/Ubuntu)

```
sudo apt update
sudo apt install git -y
```

Windows

- Download from git-scm.com
- Install with default settings

Verify:

```
git --version
```

---

5. Git Configuration

Git configuration is where you tell Git **who you are**.

So every time you make a commit, Git knows:

- Your name
- Your email

Commands:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

Check:

```
git config --list
```

---

6. Basic Git Commands

Initialize Repository

```
git init
```

Clone Repository

git clone <repo-url>

Check Status

git status

---

7. File Tracking Commands

git add file.txt : Add specific file
git add . : Add all files
git commit -m "message" : saved snapshot of your project at a specific point in time**.**

---

8. Viewing Changes

```
git log        # View history
git diff       # Show changes
```

---

9. Branching in Git

Definition

A **branch** in Git is a **separate line of development**.

It lets you work on something **without affecting the main project**

Simple Meaning

Branch = Copy of your project to work safely

Why Branching is Used?

- To add new features
- To fix bugs

- To experiment
- Without breaking the main code

Key Idea

Changes in one branch **do NOT affect** another branch (until you merge)

Important Commands

Create branch

- git branch new-feature

Switch branch

- git checkout new-feature or git switch new-feature

See branches

- git branch

What is Merge?

After finishing your work:

- git checkout main
- git merge new-feature

This combines your changes into main

Easy Analogy

Think like this:

- **main branch** = Final exam paper
- **branch** = Rough work paper

You experiment on rough paper
Then copy final answer to main

11. Remote Repository

A **remote repository** is a version of your project stored **on the internet (cloud)**.

Add remote:

git remote add origin <url>

Push:

git push -u origin main

Pull:

git pull

---

12. Git Workflow

Steps:

1. Create/modify file
2. Add (git add)
3. Commit (git commit)
4. Push (git push)

---

13. Undo Changes

- git restore file.txt # Undo changes
- git reset HEAD file.txt # Unstage file
- git reset --hard # Reset everything

---

14. Git Stash

git stash
git stash apply

Temporarily saves changes

---

15. Git Tags

git tag v1.0

Used for versions/releases

---

16. .gitignore File

Used to ignore files:

Example:

```
node_modules/
*.log
.env
```

---

17. Advantages of Git

- Distributed system
- Fast performance
- Branching is easy
- Data integrity
- Open-source

---

18. Disadvantages of Git

- Steep learning curve
- Complex commands
- Merge conflicts can be confusing